

## A Lightweight MDSD Process Applied in Small Projects

Gábor Guta\*, Wolfgang Schreiner\* and Dirk Draheim†

\*Research Institute for Symbolic Computation (RISC), Johannes Kepler University, Linz, Austria

Email: {Gabor.Guta, Wolfgang.Schreiner}@risc.uni-linz.ac.at

†Central Information Technology Services, University of Innsbruck, Austria

Email: draheim@acm.org

**Abstract**—Model driven software development, domain specific languages and other generative programming approaches have gained much attention. There is a large number of tools available, starting from simple code generators to full-blown tool suites. There are several success stories and process frameworks about applying full scale MDSD approaches, but there is no or little help for small-size projects. To fill this gap, we designed a new development process which targets small and middle size projects with limited resources. According to our experience the process is working efficiently in small projects even if they cannot afford bigger initial resource investment. In this experience report we document an industrial project in which the process was used. We present this process in the form which is applied in the first project and also discuss our experience and the limitations of the process.

**Keywords**—MDSD, Agile Development, Software Process

### I. INTRODUCTION

In this paper we describe a new lightweight model driven software development (MDSD) process applied in small development projects. We kept from the general MDSD methodology [1] just those parts that are needed for small projects. We combined these parts of the MDSD process and an iterative software development process to our new process. According to our experience this new process is working efficiently in small projects even if they are sensitive to the initial cost factor. The major contribution of this experience report is to document that (and to which extent) it was possible to apply such a process in an industrial project. We describe the process as it was applied in the first project and we also discuss the lessons learned from it.

Most of the resources available for practitioners focus on the technological aspects [2]. Books about this topic [3], [4] also give no or little hint about how MDSD can be integrated to the software development process. The process, if mentioned, is explained in the context of technological steps [5]. On the other side, several proposed methodologies target enterprise environments with sophisticated, customizable methodologies [6], [7], [8], [9]. Case studies like [10] even explicitly state that the MDA technologies are appropriate only for large organizations. Certain agile practitioners are of a similar opinion [11]. We have not found anything closer to our approach.

To successfully apply a new approach in a project there are several other factors to be considered that are independent from the theoretical soundness of the approach.

Its learning curve, its adaptability to special requirements, and the maturity of the supporting tools are probably the most crucial properties. We aimed to give a solution with the following pragmatics:

- We explicitly support the partial usage of MDSD i.e. we generate code for just those concepts of the domain model where it pays off; otherwise features are programmed in the traditional way. The advantage of this solution is that the domain model can be easily kept focused and abstract.
- Creating an own MDSD tool using an existing language saves the license and learning investment of a new tool and transformation language. Using open source software may eliminate the license cost. The mismatch between the capability of the tool and requirements of the user introduce further cost factor. According to our experience in small projects these costs outweigh the extra cost factors that arise from building their tools.
- We build the MDSD tools iteratively, which means that each new requirement in the project may lead to new requirements in the MDSD infrastructure.

In the following section, we outline the project context and our approach to solve the stated problem. Then we describe in detail the process we built on the top of the existing method used by the project team. Finally, we conclude our experience and the lessons learned.

### II. PROJECT BACKGROUND

Our customer needed MDSD process and tool support to deliver an enterprise web application for a specific domain. The first author participated in the project as consultant and contributed to development of the MDSD infrastructure to understand what kind of challenges are appear in such type of industrial projects. The deliverable application had to build on top of Microsoft's ASP.NET 2.0 technology and an MS-SQL 2000 back-end. It also had to conform to architectural guidelines for the multi-layer enterprise ASP.NET applications at that time. Readable source code and complete developer documentation was also a must, because the end customer intended to extend and maintain the delivered application in house. The project team carrying through the project was used to a traditional iterative development method, which contained elements from current *Agile methodologies* but did not conform to any of them exactly.

Informally, the requirement was to generate code from a model similar to the code generated by built-in "wizards" of Visual Studio. The point is to allow code generation without human interaction even in the case of model changes and speed up the development of the simple parts of the application. The requirements to the model driven software development approach we had to deliver were as follows:

- Most of the risks should be mitigated: The technology should not be based on immature tools and the introduction of the technology should always leave open the possibility to fall back to the traditional methodology without MDSD tools.
- The approach should not delay the schedule in general. It should have immediate business advantage.
- The approach should be cost sensitive. The purchase of a high cost MDA tool set and the extensive training of experts are not possible.
- The domain-model should be kept as simple as possible and the generated code should be human-readable and modifiable.
- The results should be reusable in other projects.

### III. THE PROCESS

In our process we distinguished two different phases: the *initial phase* and the *development phase*. The development phase contained many iterations. Figure 1 gives an overview of the process. The diagram shows the initial phase and the first two iteration phases. Presenting the iterative phase twice in the diagram makes it possible to show the connection between them. The artifacts are represented by boxes and the activities are represented by ellipses. Shading of the shapes means that they were part of the original development process. An arrow can connect an activity to an artifact or an artifact to an activity, which means the activity produces the artifact or the artifact used in the activity, respectively. The flow chart is divided by dashed lines into three "swim-lanes". The "swim-lanes" represent three different internal teams, namely: *agile development team*, *business analyst team*, and *model driven software development team*. The horizontal position of the activities express which internal teams they belong to. If an activity or an artifact is positioned between two swim-lanes, then it is a joint effort of the two teams. Under the term "internal team" we just mean a group of people who have focus on a certain type of work. These internal teams are communicating heavily with each other and have the overview over the whole project.

- The *agile development team* was responsible for the development of all non-generated software (functionality which was not represented in the domain model).
- The *business analyst team* was responsible to keep in touch with the customer and was also responsible for the requirements and the domain models.
- The *model driven software development team* was responsible for the development of the MDSD environment and the templates.

The vertical axis in Figure 1 denotes the time and the horizontal lines represents the borders of the phases. The figure shows the initial phase and two iterations such that the connections between phases are depicted. The vertical positions of the objects express how they are related in time. The arrows also provide information about the dependency of two actions. If two objects are connected and they have the same vertical position, they follow each other immediately.

Several artifacts were created and maintained during the project. We emphasize only the following two (the functions of the further artifacts follow clearly from the description of the process or used according to the common sense):

- An *MDSD vision* is a document containing the strategic design decisions related to the MDSD. It specifies which aspects of the requirement are implemented with MDSD technology. It also prescribes the abstraction level of the domain model and its scope.
- A *domain test model* is a special model which serves as a test for the generation process. It is an artificially created small model with good meta-model coverage. It is used by the developers and it does not represent any real life model.

We also define here three activities which need to be clarified to make the understanding of the the process easier:

- *The MDSD environment setup* is the activity when the code generation tool is set up according to the MDSD vision and the initial domain model. As part of this activity, the initial version of the domain meta-model and the domain test model are created.
- *Template development* is the activity by which the templates are extended and refined. This activity usually happens in three steps: the required functionality is implemented in a software prototype; the implementation is extracted and inserted into the templates; finally, the they are tested with the help of the domain test model.
- *Integration* is the activity by which the generated code and the hand-crafted software are merged. After the merge, the integration test is carried out.

The remaining terms are used in their usual sense [12], [13]. The reader may find the full definitions in [14].

#### A. Initial phase

The process started with an initial phase. There were three groups of activities in this phase:

- The business analysts team and the MDSD team extracted the domain model by a joint effort. Domain experts from the customers also contributed.
- The agile development team elaborated the architecture of the software. By the end of the initial phase they shipped a sufficiently stable prototype to the MDSD team which started to develop the templates.
- The MDSD team set up the initial version of the code generator core. This team was also familiar with the technology environment used by the agile

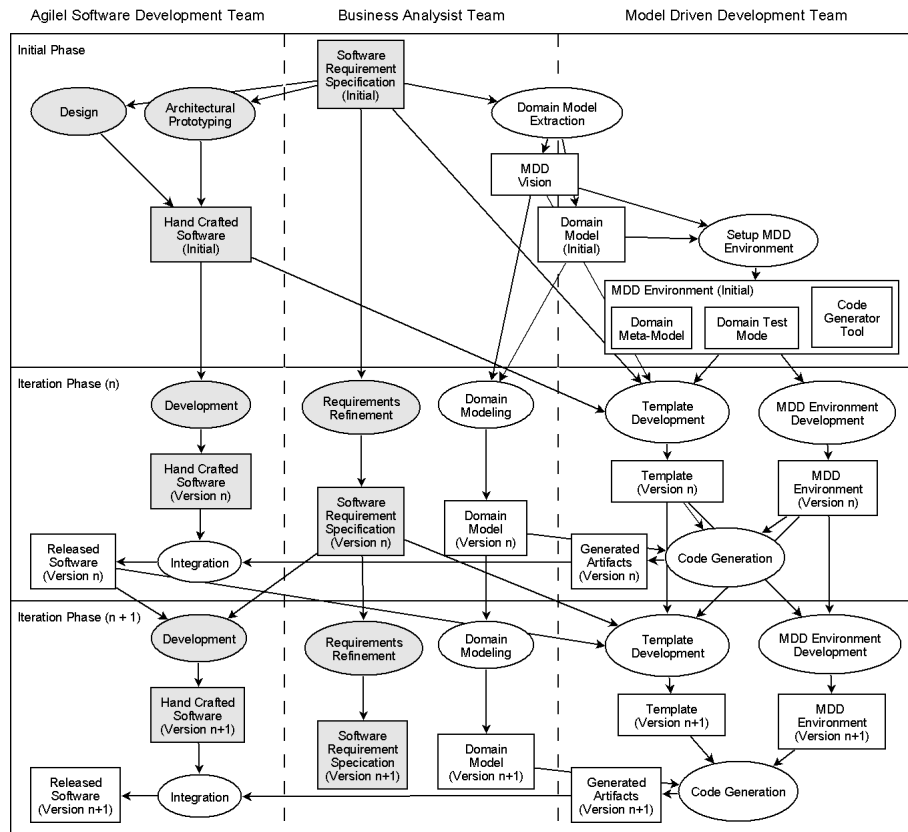


Figure 1. The Process

software team and took over the architecture from the prototypes.

By the end of the initial phase we had the first version of the domain model represented in Excel spreadsheets, an informal description of the domain meta-model, a working version of the code generation tool and a part of the domain model as a domain test model. Our customer successfully used the first generated code to demonstrate to the end customer the navigation structure of the application.

### B. Development phase

During an iteration of the development phase, the following activities were carried through in parallel:

- The business analyst team refined and extended the domain model. The changes in the domain model did not affect the developers, so the model could be changed until the code generation activity started. This team kept in touch with the customer; its responsibility was to handle the change-requests of the requirements.
- The agile development team developed the non-generated part of the software. The customization of the generated parts and the integration of the custom components were also the responsibility of this team. In the end of the iteration this team received the generated artifacts and carried through the integration. There were always cases when the newly

generated artifacts needed manual modifications or interface mismatches had to be corrected. To ensure the success of the integration also integration tests and regression tests were performed. Critical bugs were corrected during the integration and the whole test activity was repeated.

- The MDSD development team was busy with the template development and the MDSD environment development. In the end of the iteration this team produced the generated artifacts from the domain model with the help of the latest MDSD environment and templates. The enhancement of the MDSD environment was usually orthogonal to the template development.

The template development was the most complex and critical activity of the process. Typically for an iteration of the process several features were planned to be implemented iteratively in the frame of this activity. The implementation of a feature started with its design, then the feature was coded and tested. During the implementation the generated development artifacts were extended.

The generated development artifacts were parts of the runnable application which was generated by the latest version of the templates, of the code generation tool, and of the domain test model. The implemented feature was extracted from the source code and inserted into the templates with the necessary modifications. The generated development artifacts were re-created by the modified

templates. The result typically contained small errors, which were detected when the result was built and tested. The test was carried out with the test cases of the original feature implementation. The templates were modified until no more errors occurred during the test of the generated development artifacts. Then the feature was considered complete and the team moved on to implement the next one.

#### IV. CONCLUSION AND OUTLOOK

Model driven software development and domain specific languages are active research areas both for the academic and the industrial communities. In this paper, we have described a lightweight MDSD process which was successfully applied in industrial projects; we provide a comprehensive description of the minor details which need to be understood for the real world projects [14]. Our approach is not only usable for small and mid-size projects, but can be also used as preparatory step before the introduction of a heavyweight MDSD approach [3], [5].

Our project was considered by our customer successful in comparison with its previous agile projects. However, it is not clear how to evaluate the project objectively. It may be judged according to basic project metrics, but there are no such data available from similar projects. According to the project culture of the company, no formal size estimation was carried through before the development and the project metrics were also recorded barely. Here we just include these metrics which were considered trustworthy according to the internal project communication. In the final release the total amount of source code developed was approximately 300 thousand lines of code (KLOC). This contained 200 KLOC of generated code, which was produced from 0.9 KLOC lines of XML specification by 5 KLOC of XSLT. The project team had 8 members and the net effort to complete the project was around 25 man-months. There were only minor issues that went wrong during the project, but we managed to fix them in time.

At the end of our project we also experienced limitations of our approach. We think that it does not scale up further without dealing with the problem of manual merging of the different versions of artifacts. Automated merging support is needed for those situations in which it is not possible to keep the number of these artifacts low. Unfortunately, there is not much research documented in literature on merging artifacts on different abstraction levels; this problem is also still an open issue in most state of the art tools.

New transformation languages and tool suites will definitely ease the MDSD, but it needs to be evaluated to what extent. We found several issues that need further investigation, like semantics of merging artifacts or optimization of the test procedures. Currently, we are investigating the applicability of formal methods in this area.

#### REFERENCES

[1] B. Selic, "The Pragmatics of Model-Driven Development," *Software, IEEE*, vol. 20, no. 5, pp. 19–25, Sept.-Oct. 2003.

- [2] "Code Generation Information for the Pragmatic Engineer," <http://www.codegeneration.net/>.
- [3] D. S. Frankel, *Model Driven Architecture: Applying MDA to Enterprise Computing*. Wiley, 2003.
- [4] J. Greenfield, K. Short, S. Cook, and S. Kent, *Software Factories: Assembling Applications with Patterns, Models, Frameworks, and Tools*. Wiley, 2004.
- [5] A. G. Kleppe, J. Warmer, and W. Bast, *MDA Explained: The Model Driven Architecture: Practice and Promise*. Addison-Wesley, 2003.
- [6] F. Chitforoush, M. Yazdandoost, and R. Ramsin, "Methodology Support for the Model Driven Architecture," in *APSEC '07: Proceedings of the 14th Asia-Pacific Software Engineering Conference (APSEC'07)*. IEEE CS, 2007, pp. 454–461.
- [7] A. Gavras, M. Belaunde, L. F. Pires, and J. P. A. Almeida, "Towards an MDA-Based Development Methodology," in *EWSA*, ser. LNCS, vol. 3047. Springer, 2004, pp. 230–240.
- [8] J. X. Mansell, A. Bediaga, R. Vogel, and K. Mantell, "A Process Framework for the Successful Adoption of Model Driven Development," in *ECMDA-FA*, ser. LNCS, vol. 4066. Springer, 2006, pp. 90–100.
- [9] M.-P. Gervais, "Towards an MDA-Oriented Methodology," in *COMPSAC*. IEEE CS, 2002, pp. 265–270.
- [10] V. Kulkarni and S. Reddy, "Introducing MDA in a Large IT Consultancy Organization," in *APSEC*. IEEE Computer Society, 2006, pp. 419–426.
- [11] A. Uhl and S. W. Ambler, "Point/Counterpoint: Model Driven Architecture Is Ready for Prime Time / Agile Model Driven Development Is Good Enough," *IEEE Software*, vol. 20, no. 5, pp. 70–73, 2003.
- [12] C. Larman, *Agile and Iterative Development: A Manager's Guide*. Addison-Wesley, 2003.
- [13] A. S. Koch, *Agile Software Development: Evaluating The Methods For Your Organization*. Artech House, 2004.
- [14] G. Guta, B. Szasz, and W. Schreiner, "A Lightweight Model Driven Development Process based on XML Technology," RISC Report Series, University of Linz, Austria, Tech. Rep. 08-01, March 2008.